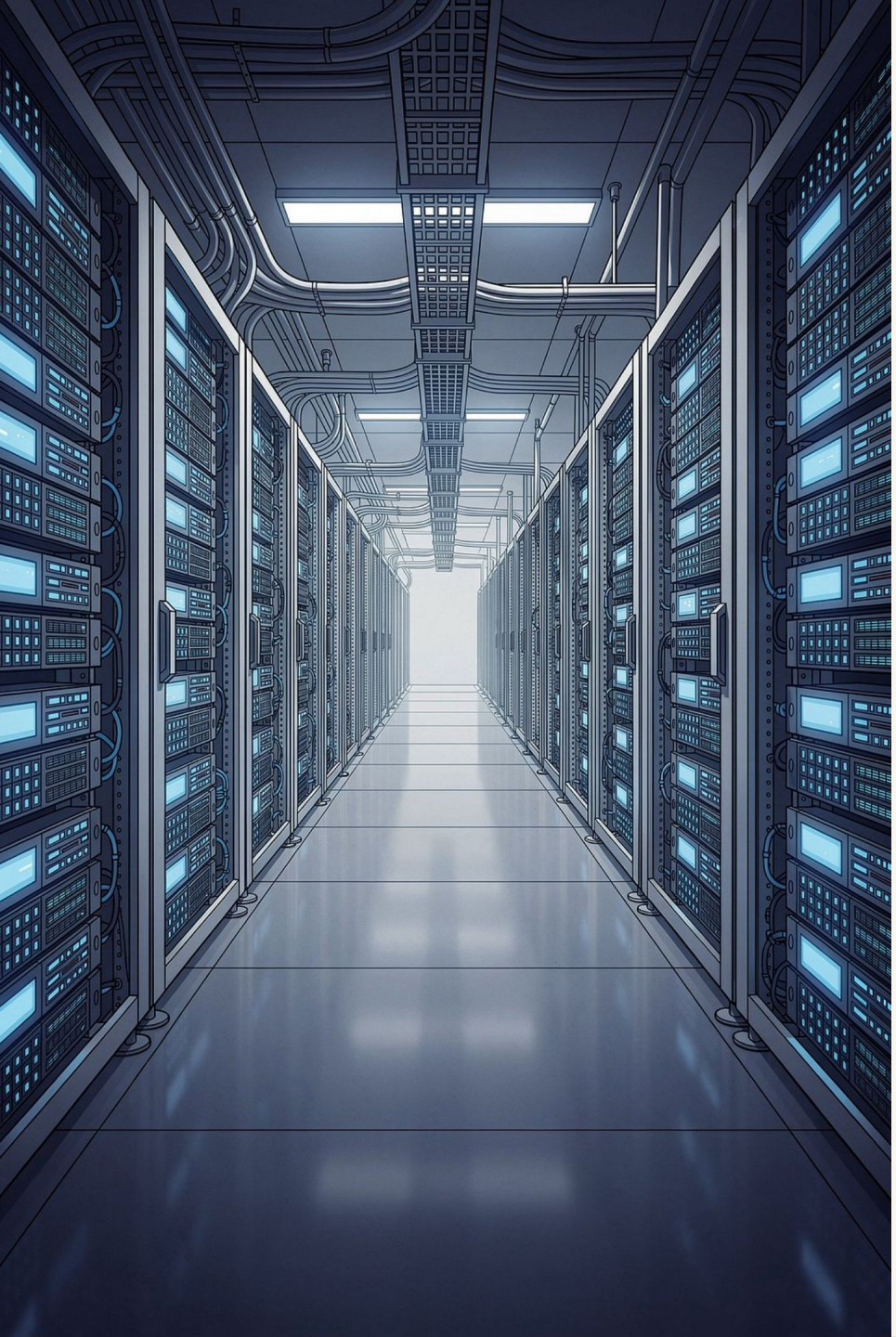


High Availability Validation

LAB PART 2

A hands-on Kubernetes lab for intermediate administrators — validating that applications remain available even when worker nodes fail.

LAB OVERVIEW

What You'll Learn

Business Scenario

ABC Retail requires its applications to remain available even if a worker node fails. This lab validates that Kubernetes scheduling policies and self-healing mechanisms meet that requirement.

Key Learning Outcome: High Availability depends on proper scheduling policies and sufficient worker node capacity.

Lab Objectives

01

Deploy

A highly available application with 3 replicas

02

Configure

Pod Anti-Affinity to spread replicas across nodes

03

Simulate

Worker node failure and observe self-healing

04

Verify

Application availability and recovery behavior

TASK 1

Deploy the Production Application

Deployment Spec

- 3 Replicas
- RollingUpdate Strategy

Resource Constraints

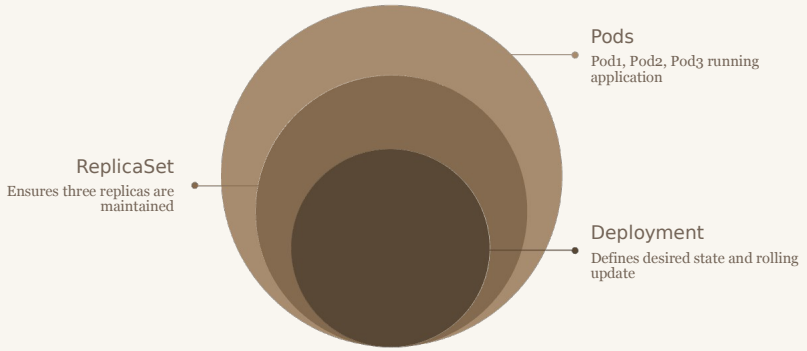
- CPU & Memory Requests
- CPU & Memory Limits

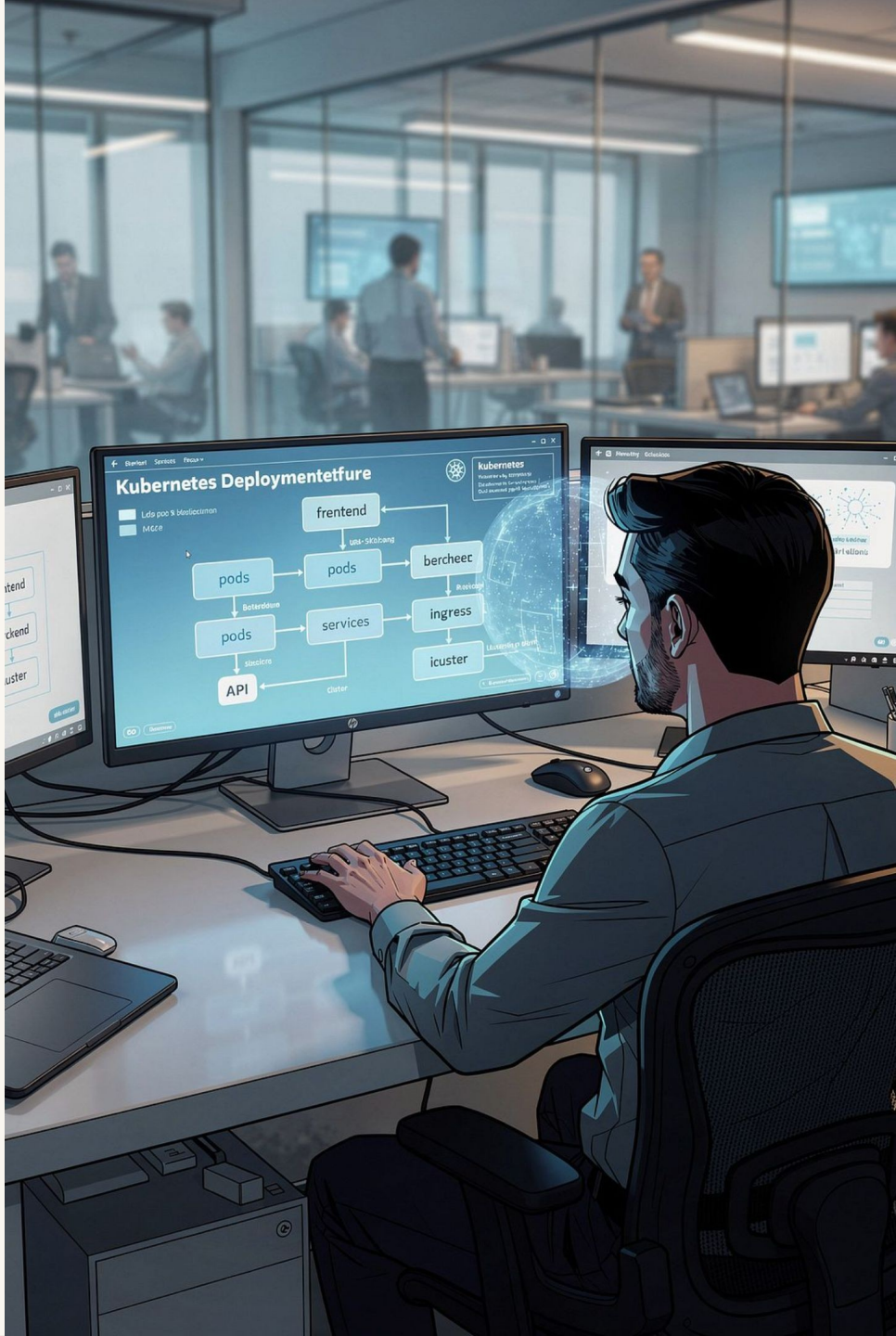
Networking

- NodePort Service
- Application Accessible

Expected Result

- 3 Running Pods
- ReplicaSet managed

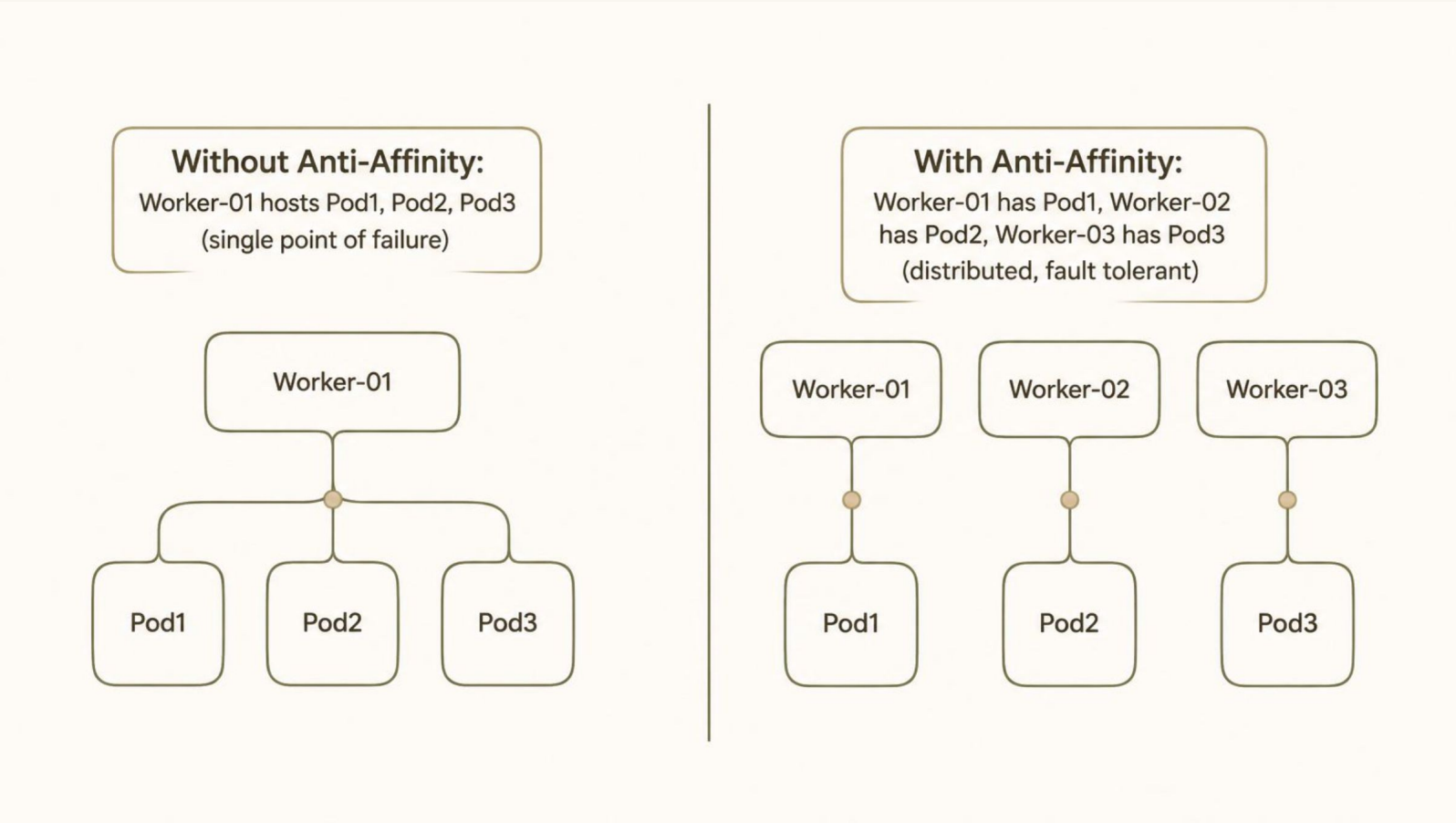




TASK 2

Configure Pod Anti-Affinity

Ensure application replicas are distributed across worker nodes to prevent a single node failure from impacting all replicas.



Prevent SPOF

No single node failure can take down all replicas

Improve HA

Replicas spread across the cluster for resilience

Fault Tolerance

Workload survives individual node outages

TASK 3

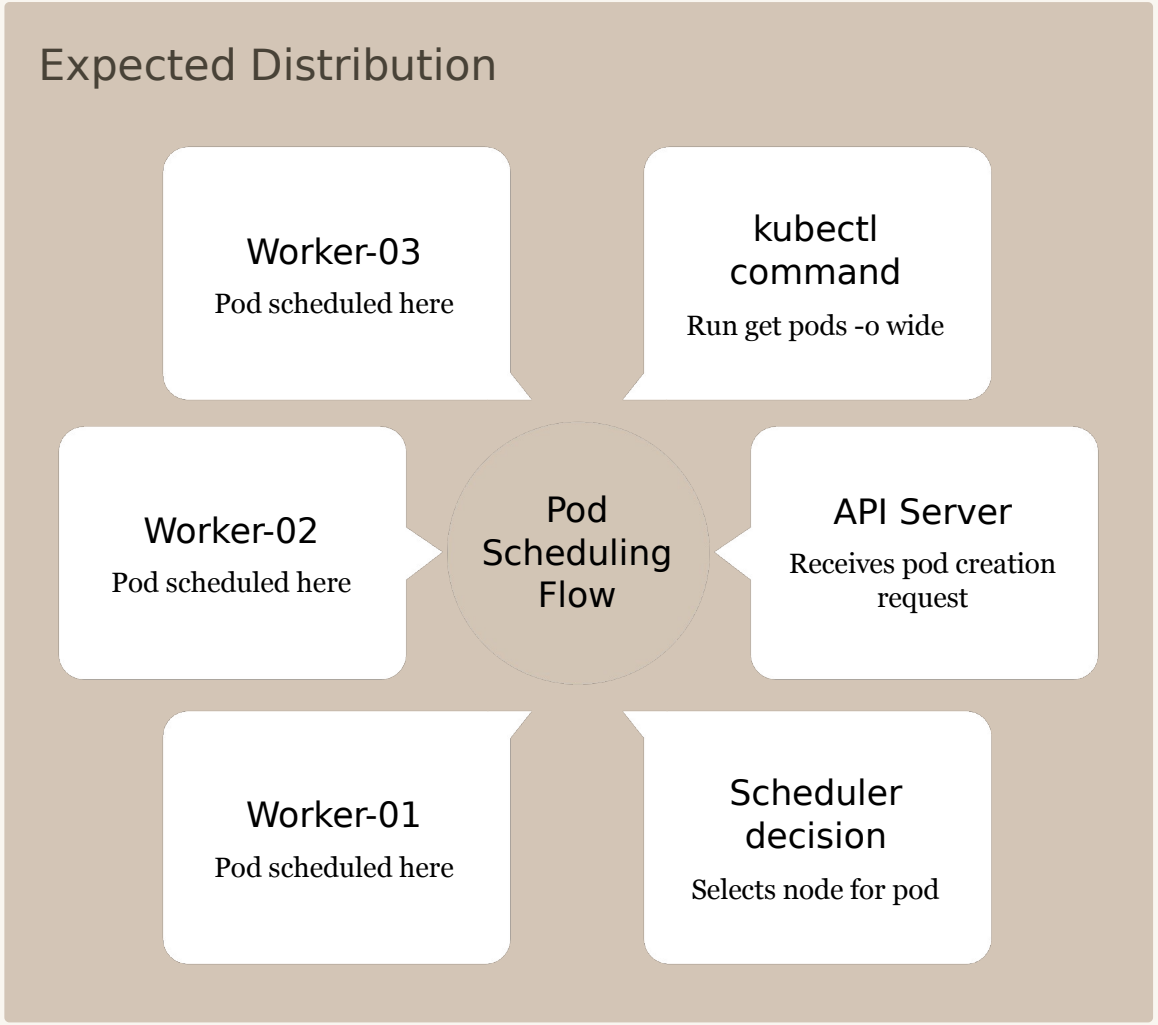
Verify Pod Scheduling

Validate Pod Placement

Run `kubectl get pods -o wide` to inspect scheduling decisions made by the Kubernetes Scheduler.

- Which worker hosts each pod?
- Are replicas spread across nodes?
- Which scheduling policy influenced placement?

Expected Distribution



TASK 4

Simulate Worker Node Failure

1. **Ready**
   All nodes healthy, pods running normally
2. **Shutdown**
   Worker-01 powered OFF by instructor
3. **NotReady**
   Node status changes, detected by control plane
4. **Pods Lost**
   Pods enter Unknown / Terminating state
5. **Rescheduled**
   Deployment recreates pods on remaining workers

Monitor with: `kubectl get nodes` and `kubectl get pods -w`

TASK 5

Validate High Availability

Verification Checklist

- Deployment Status
- Replica Count
- Pod Health
- Service Accessible

Discussion Points

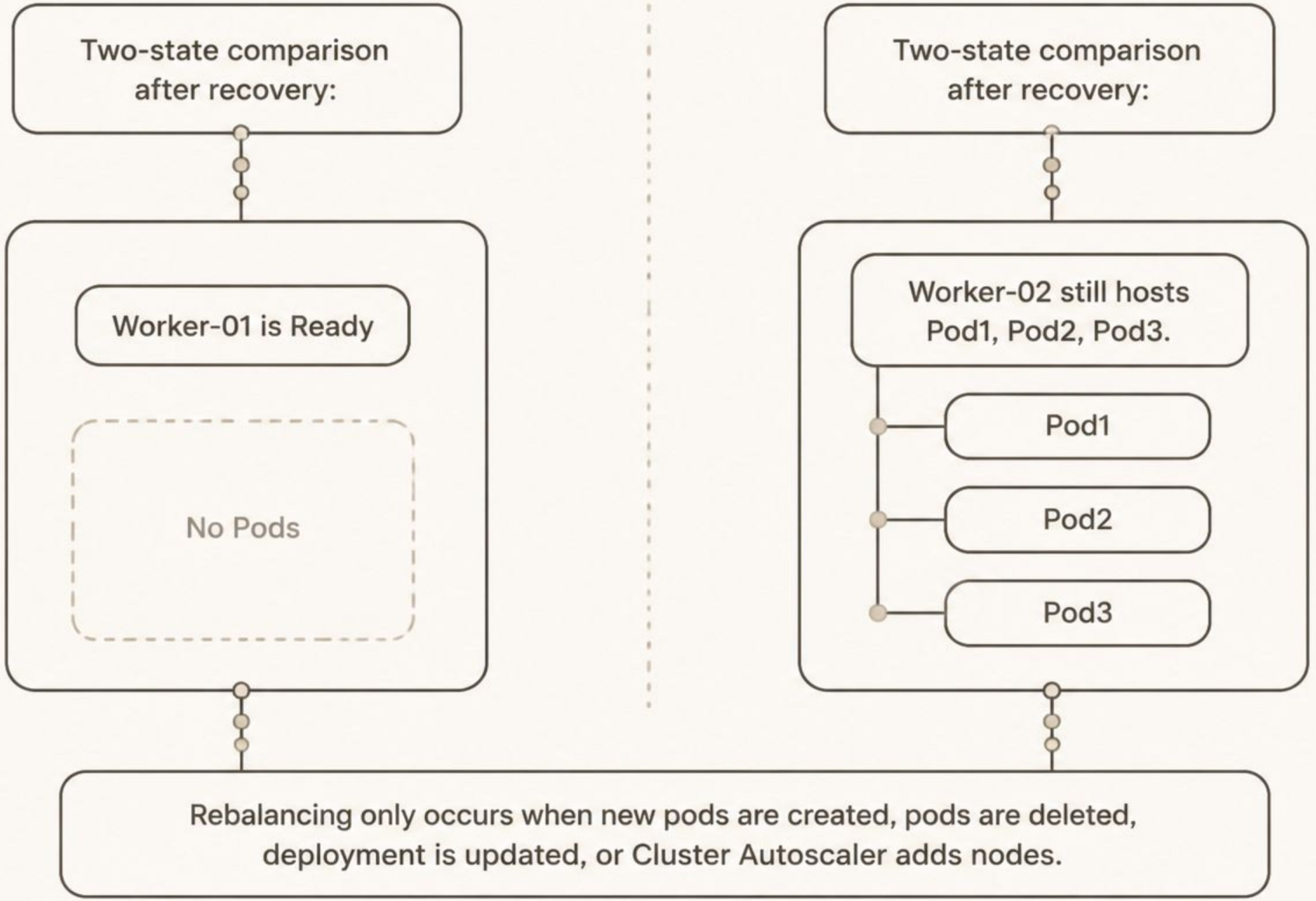
- Which replicas recovered, and why?
- Which scheduling rules affected recovery?
- What happens if only one worker remains?

High Availability depends on the number of available worker nodes at the time of rescheduling.

TASK 6

Worker Recovery — No Auto-Rebalance

Power **ON** Worker-01 and observe the node return to `Ready` state. However, Kubernetes **does not rebalance pods automatically.**



New Pods Created

Pods Deleted

Deployment Updated

Autoscaler Adds Nodes

LAB SUMMARY

High Availability Checklist

Isolation

- Workloads isolated using Labels
- Taints & Tolerations configured
- Monitoring on dedicated nodes

Resources

- CPU & Memory Requests defined
- Pod Anti-Affinity configured

Resilience

- Worker failure detected
- Pods recreated on healthy workers
- Application remains available

Key Takeaways

Pod Anti-Affinity improves workload distribution

Spreading replicas across nodes prevents a single point of failure.

Deployments provide self-healing

The Scheduler honors labels, taints, tolerations, affinity, and resource constraints when recreating pods.

Worker recovery does not rebalance pods

Running pods stay on their current nodes; rebalancing only occurs on new scheduling events.

More workers = greater resilience

Additional worker nodes provide more fault tolerance and scheduling flexibility.